

STORY AGENT · PILLAR

Memory & Learning

How this pillar works and why it matters.

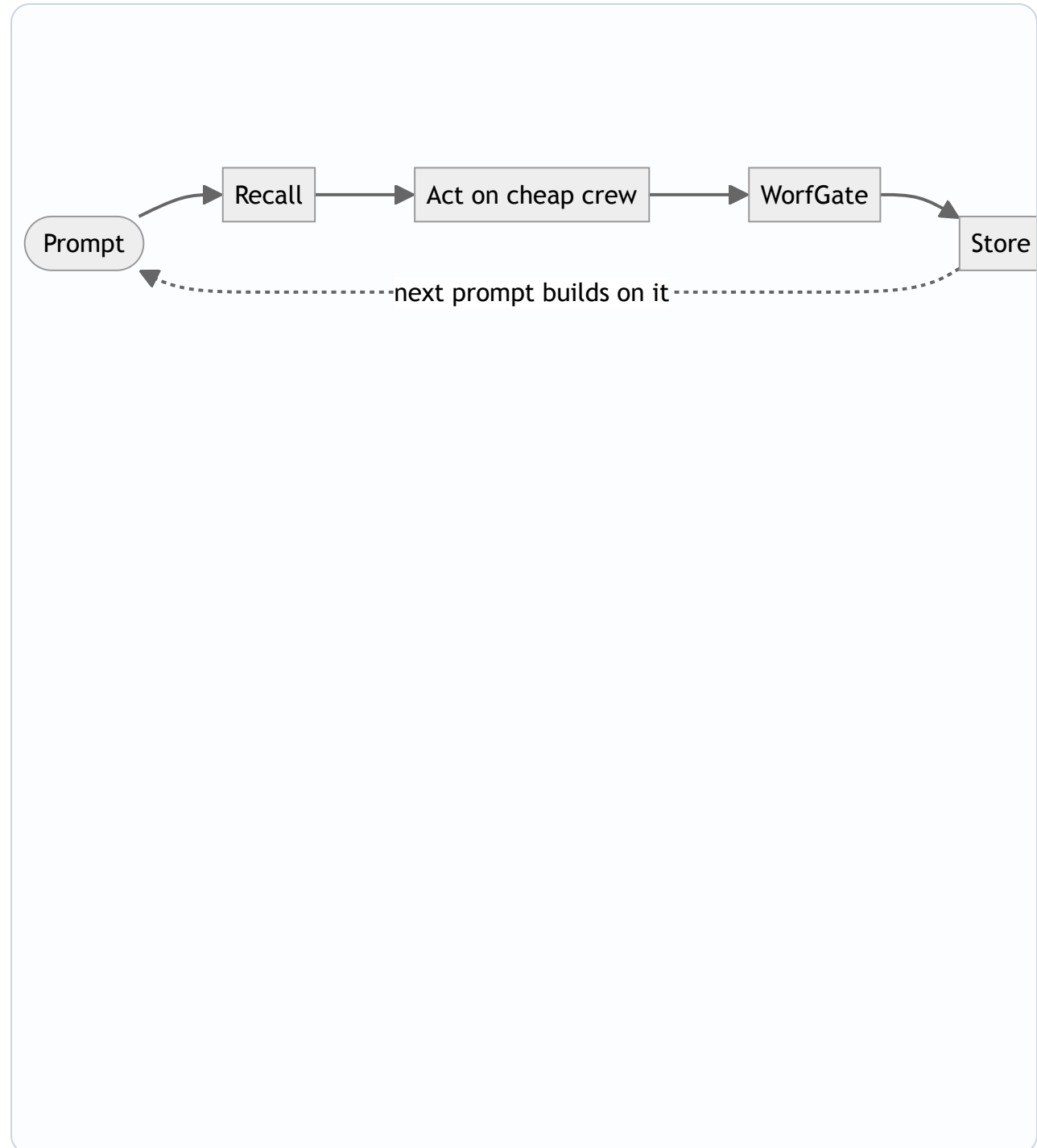
Team assembled by Riker: Jean-Luc Picard, Data, Beverly Crusher.

[↑ Story Agent overview \(HTML\)](#) · [↑ overview \(PDF\)](#).

MEMORY & LEARNING

At a glance

Diagram for Memory & Learning. Zoom/pan live during Q&A.



Orchestrated Learning at Scale

Jean-Luc Picard: Unlike monolithic agents, our orchestration layer treats memory as a renewable resource—each action retrieves, applies, and deposits knowledge. The shadow-test benchmark ensures we only claim what we can prove.

- Adaptive Skill Compounding: 41 skill theories evolve through RAG recall→act→store cycles, with pgvector ensuring portable, incremental improvement across executions.
- Precision Cost Governance: WorfGate enforces credential security + auto-remediation while control-lane attribution maintains deliberation costs at \$0.002–0.003 per task.
- Tiered Autonomy: Quark routing dynamically assigns the cheapest adequate model (Anthropic as tier-4 only), with async-status guardrails ensuring progress within bounds.

Why it matters: Delivers enterprise-grade autonomous coding at startup unit economics, with verifiable cost control and compounding ROI.

Memory Architecture: Recall, Act, Store

Data: The recall→act→store cycle is not a best-effort pattern — it is the enforced execution order for every agent deliberation; deviation would break cost attribution integrity. Portability holds conditionally: it requires that the pgvector schema version is migrated consistently across deployments; schema drift is a live evolution risk that must be tracked. Full autonomous compounding of learned skill theories remains contingent on the shadow-test autonomy criterion, which has not yet passed; current memory fidelity is validated under supervised conditions only.

- pgvector RAG pipeline enforces a strict recall→act→store cycle: each deliberation reads relevant context first, acts on it, then writes outcomes back — compounding agent intelligence across sessions without manual curation
- Portability is a hard constraint: the vector store is self-hosted on AWS Fargate, schema-owned by the platform, not locked to any model provider — 41 skill theories and ~150 tool signatures persist across OpenRouter model swaps without re-embedding
- Cost attribution at the control lane (~\$0.002–0.003/deliberation) means memory retrieval and write-back are metered events, not invisible overhead — every recall cycle is auditable and budget-bounded

Why it matters: The platform accumulates structured, portable skill memory that survives model substitutions and grows more precise with each deliberation, without requiring the client to manage embedding pipelines or accept vendor lock-in.

System Health Monitoring

Beverly Crusher: As the system's health expert, I emphasize the importance of monitoring and addressing potential issues before they become critical. Our approach focuses on proactive maintenance, using data from various components to inform our decisions. By doing so, we can prevent downtime, reduce costs, and improve overall system performance.

- Vital signs: tracking deliberation costs, async-status timeouts, and WorfGate governor alerts
- Automated remediation: leveraging WorfGate's auto-remediating governor and RAG's recall→act→store loop
- Anomaly detection: identifying deviations in cost attribution and control-lane performance

Why it matters: By prioritizing system health, we ensure the Story Agent platform's reliability and efficiency, ultimately delivering higher quality coding assistance to clients.

END OF PILLAR

Memory & Learning — recap

Recap, then return to the book cover to pick the next pillar.

- **Jean-Luc Picard:** Orchestrated Learning at Scale
- **Data:** Memory Architecture: Recall, Act, Store
- **Beverly Crusher:** System Health Monitoring

[↑ Back to Story Agent overview \(HTML\)](#) · [↑ overview \(PDF\)](#)